

COMPACTRAG: Reducing LLM Calls and Token Overhead in Multi-Hop Question Answering

Hao Yang*

State Key Laboratory for Novel
Software Technology, Nanjing
University
Suzhou, Jiangsu, China
howyoung80@163.com

Zhiyu Yang*

Erik Jonsson School of Engineering
and Computer Science, University of
Texas at Dallas
Richardson, Texas, USA
zhiyu.yang@utdallas.edu

Xupeng Zhang*

Isoftstone Information Technology
(Group) Co.,Ltd.
Beijing, China
lagelangpeng@gmail.com

Wei Wei

College of Electronic and Information
Engineering, Tongji University
Shanghai, China
2510856@tongji.edu.cn

Yunjie Zhang

School of Electronic Information,
Central South University
Changsha, Hunan, China
Zhangyj@csu.edu.cn

Lin Yang[†]

State Key Laboratory for Novel
Software Technology, Nanjing
University
Suzhou, Jiangsu, China
linyang@nju.edu.cn

Abstract

Retrieval-augmented generation (RAG) has become a key paradigm for knowledge-intensive question answering. However, existing multi-hop RAG systems remain inefficient, as they alternate between retrieval and reasoning at each step, resulting in repeated LLM calls, high token consumption, and unstable entity grounding across hops. We propose COMPACTRAG, a simple yet effective framework that decouples offline corpus restructuring from online reasoning.

In the offline stage, an LLM reads the corpus once and converts it into an *atomic QA knowledge base*, which represents knowledge as minimal, fine-grained question-answer pairs. In the online stage, complex queries are decomposed and carefully rewritten to preserve entity consistency, and are resolved through dense retrieval followed by RoBERTa-based answer extraction. Notably, during inference, the LLM is invoked only twice in total—once for sub-question decomposition and once for final answer synthesis—regardless of the number of reasoning hops.

Experiments on HOTPOTQA, 2WIKIMULTIHOPQA, and MUSEQUE demonstrate that COMPACTRAG achieves competitive accuracy while substantially reducing token consumption compared to iterative RAG baselines, highlighting a cost-efficient and practical approach to multi-hop reasoning over large knowledge corpora. The implementation is available at <https://github.com/How-Young-X/CompactRAG>.

*All authors contributed equally to this research.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792512>

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; • **Computing methodologies** → *Natural language processing*.

Keywords

Retrieval-augmented generation, Multi-hop question answering, Efficient reasoning

ACM Reference Format:

Hao Yang, Zhiyu Yang, Xupeng Zhang, Wei Wei, Yunjie Zhang, and Lin Yang. 2026. COMPACTRAG: Reducing LLM Calls and Token Overhead in Multi-Hop Question Answering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792512>

1 Introduction

Retrieval-Augmented Generation (RAG) [17] is now a standard approach for knowledge intensive NLP. RAG combines explicit retrieval with the generation and reasoning capacity of large language models (LLMs). This combination works well for question answering. LLMs can produce factual, grounded answers by retrieving relevant passages [2, 6, 11, 16, 21]. However, multi-hop question answering (MHQA) [9, 18, 33, 34, 37] is more challenging. A MHQA query requires integrating evidence from multiple documents. Conventional RAG pipelines face three recurring problems in this setting. First, efficiency degrades as reasoning hops increase [5, 12, 33]. Second, retrieved context often contains redundant information [15, 26, 27, 29]. Third, maintaining factual consistency across hops is difficult [8, 14, 41]. These challenges are central to web scale information access, where large and heterogeneous knowledge sources must be efficiently retrieved, represented, and reasoned over by LLMs.

Recent work implements iterative retrieval generation cycles for multi-hop reasoning. Examples include SELF-ASK [24], IRCOT [35], and ITER-RETGEN [28]. These methods alternate between retrieval and LLM reasoning. At each step, the model retrieves passages guided by prior reasoning. This design improves factual coverage and yields explicit reasoning chains. It also increases the number of

LLM invocations. As a result, token usage and latency grow with hop depth. This growth raises computational cost and limits scalability. Multi-hop question decomposition can also harm retrieval accuracy, a common failure mode is *entity drift*. A decomposed sub-question may lose its explicit entity mention. For example, “Where was the scientist who discovered penicillin born?” can be split into “Who discovered penicillin?” and “Where was *he* born?” The second sub-question lacks an explicit entity and may retrieve unrelated documents, producing inconsistent results [42]. Prior work documents related failures when decomposition is imprecise [23, 42].

A large body of follow up work attempts to mitigate these issues by refining the retrieval–reasoning interaction. HOPRAG [19] and LEVELRAG [40] introduce hierarchical or logic-aware retrieval to enhance reasoning paths, yet still require multiple LLM invocations per hop. DUALRAG [3] and GENGROUND [30] employ iterative “generate then ground” loops to couple generation and retrieval, which increases computational overhead. Q-DREAM [39] dynamically optimizes sub-question semantics in a learned retrieval space, but depends on several LLM-driven refinement stages. CHAINRAG [42] builds a sentence-level graph to preserve entity continuity and alleviate entity drift, at the cost of heavy graph traversal and multiple reasoning–retrieval cycles. Other works leverage internal model signals such as attention entropy or decoding uncertainty [7, 13, 25, 31, 38], but these approaches require access to non-public model activation, limiting their deployability. Finally, EFFICIENTRAG [43] reduces online LLM involvement via lightweight retriever modules, yet still operates directly over raw corpus passages, leaving substantial redundancy in retrieved context.

We propose COMPACTRAG, a simple and practical alternative. CompactRAG separates corpus processing from online inference. Offline, an LLM reads the corpus once and constructs an *atomic QA knowledge base*. These QA pairs are concise, fact-level units that reduce redundancy and better align with question semantics [32]. Online, a complex query is decomposed into dependency-ordered sub-questions. Each sub-question is resolved using lightweight modules for retrieval, answer extraction, and question rewriting. The main LLM is invoked only twice per query: once for decomposition and once for final synthesis. This fixed two-call design makes LLM usage independent of hop depth. The offline step incurs a one-time cost. That cost is amortized as user queries accumulate.

Contributions. Our work makes three main contributions. First, we analyze scalability issues in iterative RAG pipelines, showing how token consumption and LLM calls grow with reasoning depth. Second, we introduce COMPACTRAG, a two-call RAG framework that uses an offline atomic QA knowledge base and lightweight online modules to enable efficient multi-hop inference. Third, we evaluate CompactRAG on HOTPOTQA, 2WIKIMULTIHOPQA, and MUsIQUE, demonstrating competitive accuracy along with significant reductions in inference token usage compared to strong iterative baselines.

2 Related Work

We review related work in three main areas: (1) multi-hop question answering and iterative retrieval–reasoning pipelines, (2) structured and corpus-level retrieval enhancement, and (3) efficiency oriented and adaptive retrieval strategies. Our discussion highlights

how COMPACTRAG differs from these paradigms by decoupling reasoning from retrieval through an offline–online architecture.

2.1 Multi-hop QA and Iterative Retrieval–Reasoning Pipelines

Multi-hop question answering benchmarks such as HOTPOTQA [37], 2WIKIMULTIHOPQA [9], and MUsIQUE [34] have driven research on compositional reasoning across documents. Early retrieval augmented approaches treat reasoning as a sequence of retrieval and generation steps. SELF-ASK [24] explicitly decomposes questions into sub-questions that are answered iteratively, using the model’s own intermediate outputs as guidance. IRCOT [35] interleaves retrieval with a chain-of-thought process [36], allowing reasoning traces to refine retrieval queries dynamically. ITER-RETRAGEN [28] further integrates iterative retrieval and generation, where each model response serves as a context for the next retrieval round. While these systems enhance factual completeness and interpretability, their reliance on repeated LLM invocations makes computational cost scale linearly with reasoning hops. Each iteration expands the prompt with retrieved passages, leading to excessive token consumption and increased latency. Moreover, automatic sub-question decomposition can suffer from *entity drift* [23, 42], where referential grounding is lost (e.g., “Where was *he* born?”), degrading retrieval precision. COMPACTRAG eliminates these iterative dependencies by executing retrieval and reasoning separately, using fixed-cost local modules for sub-question resolution.

2.2 Structured and Corpus-level Retrieval Enhancement

Beyond iterative pipelines, several studies improve retrieval grounding by introducing explicit structure or corpus level representations. HOPRAG [19] constructs paragraph graphs linking documents through logical dependencies, enabling LLM-guided traversal across hops. LEVELRAG [40] employs a hierarchical planner that combines sparse, dense, and web based retrieval to support multi-hop reasoning. DUALRAG [3] and GENGROUND [30] couple generation and retrieval through dual or generate then ground loops, progressively refining sub-queries. However, these designs require multiple LLM calls for reasoning validation and query reformulation, limiting efficiency. Q-DREAM [39] learns a dynamic retrieval space aligned to sub-question semantics using LoRA-tuned modules, while CHAINRAG [42] constructs sentence-level graphs to maintain entity continuity and mitigate lost-in-retrieval errors. Although such structures improve reasoning fidelity, they often entail costly graph traversal, embedding computation, and repeated model inference.

Another direction focuses on corpus preprocessing. EFFICIENTRAG [43] introduces lightweight modules—*Labeler*, *Tagger*, and *Filter*—to reduce online LLM calls, but it still retrieves over raw, redundant passages. Recent studies [32] observe that LLM-generated text aligns more closely with the query’s semantic space and thus serves as a more compact and expressive retrieval unit. Inspired by this, COMPACTRAG performs offline corpus restructuring into atomic QA pairs. This produces semantically complete, redundancy-free, and fact-centric knowledge units that support fine-grained

reasoning. Unlike prior structural frameworks, CompactRAG requires no online graph traversal or dynamic refinement, maintaining retrieval efficiency and stable accuracy.

2.3 Efficiency and Adaptive Retrieval Strategies

A complementary line of research improves efficiency through adaptive retrieval or model-aware decision mechanisms. DIORE [7], SEAKR [38], and DRAGIN [31] propose adaptive retrieval-augmented generation methods that monitor model internal signals, such as entropy, gradient variance, or decoding uncertainty to determine when to retrieve additional context. Active RAG [13] and Entropy-Based Decoding [25] follow similar strategies, activating retrieval only when confidence drops. Although effective in reducing redundant retrievals, these systems require access to hidden activations or attention scores, which are typically unavailable in closed weight LLMs, restricting their practicality. In contrast, COMPACTRAG achieves comparable efficiency gains through architectural design rather than internal signal access. Its offline-online separation amortizes reasoning cost across queries: the knowledge base is built once offline, and each query requires only lightweight retrieval and two fixed LLM calls online. This design ensures predictable cost, scalability, and compatibility with open or closed LLMs.

Summary. Existing RAG systems trade off reasoning accuracy, retrieval precision, and computational efficiency. Iterative pipelines improve factual reasoning but scale poorly with hop depth; graph-based and dynamic retrieval methods enhance grounding but require complex online computation; internal signal approaches remain difficult to deploy. COMPACTRAG reconciles these limitations by precomputing atomic QA representations offline and performing reasoning through modular, low cost components online. This results in a scalable and token efficient framework for multi-hop reasoning, achieving a favorable balance between accuracy and efficiency.

3 Methodology

The goal of COMPACTRAG is to reduce corpus redundancy, minimize token consumption during complex reasoning, and decrease the number of LLM calls required for multi-hop question answering. To this end, we decompose the reasoning process into two stages: (1) an *offline corpus preprocessing stage*, which constructs a concise and structured QA knowledge base, and (2) an *online reasoning stage*, which efficiently retrieves and composes relevant atomic QA pairs without repeated LLM invocations.

In the offline stage, the raw corpus is processed once by an LLM to generate a compact set of atomic QA pairs, removing noise and redundancy. In the online stage, a complex question is decomposed into a dependency graph of sub-questions, which are then iteratively resolved through retrieval over the QA knowledge base. The retrieved evidence is aggregated for a single final LLM call that synthesizes the final answer. This design keeps the number of LLM calls fixed and ensures efficiency even for deep multi-hop reasoning chains.

3.1 Offline Stage

In the offline preprocessing stage, COMPACTRAG employs an LLM to transform the raw corpus into a structured and compact *atomic QA knowledge base*. This process is performed once prior to inference, aiming to eliminate redundancy while preserving essential factual information in a form directly aligned with downstream query semantics. Inspired by prior findings [32] that LLM-generated representations tend to align more closely with the semantic space of natural queries, we prompt the LLM to read each document and reformulate its content into a set of *atomic QA pairs*. Each pair expresses a single factual statement with minimal granularity, ensuring non-overlapping information units suitable for multi-hop composition. Before generation, entities within the corpus are automatically annotated using SpaCy¹, and these entities are explicitly enforced in the generation prompt (see Appendix A). This constraint guarantees semantic completeness and prevents omission of key referential elements. An overview of this corpus to QA transformation is illustrated in Figure 1.

Dense retrieval over atomic QA knowledge. After generation, each atomic QA pair is embedded into a shared semantic space using dense retrieval representations. Unlike sparse lexical retrieval methods such as BM25, dense retrieval captures contextual and semantic similarity beyond surface word overlap, which is particularly critical for multi-hop reasoning where sub-questions often differ lexically from supporting knowledge. To maximize the semantic coherence between questions and answers, the question (q) and answer (a) components of each pair are concatenated into a single text segment $[q; a]$ before encoding. This joint representation preserves the full factual scope of each unit, allowing the retriever to index both the intent expressed in the question and the corresponding factual content in the answer. During online inference, the same encoder retrieves top- k relevant QA pairs for each sub-question based on embedding similarity, enabling compact and semantically aligned evidence retrieval from the preprocessed knowledge base.

3.2 Online Stage

As illustrated in Figure 2, the online reasoning process begins by decomposing a complex multi-hop question into a dependency-ordered set of sub-questions. One sub-question’s resolution may depend on the answer to a preceding one. Existing iterative RAG systems perform retrieval and reasoning alternately, invoking the LLM at every step to maintain accuracy, but at the cost of excessive computation and token usage. In contrast, COMPACTRAG leverages the atomic QA knowledge base to decouple retrieval from reasoning entirely. Two lightweight transformer-based modules are introduced—an *Answer Extractor* and a *Sub-Question Rewriter*. These modules enable multi-hop retrieval without involving the LLM, thereby reducing computational overhead and preventing *entity drift* across hops.

3.2.1 Multihop Question Decomposition. Given a user query Q , the system first decomposes it into a sequence of sub-questions $\{q_1, q_2, \dots, q_n\}$ organized in a dependency graph $\mathcal{G}$. Each directed edge $q_i \rightarrow q_j$ indicates that the answer to q_i is required to resolve

¹<https://spacy.io>

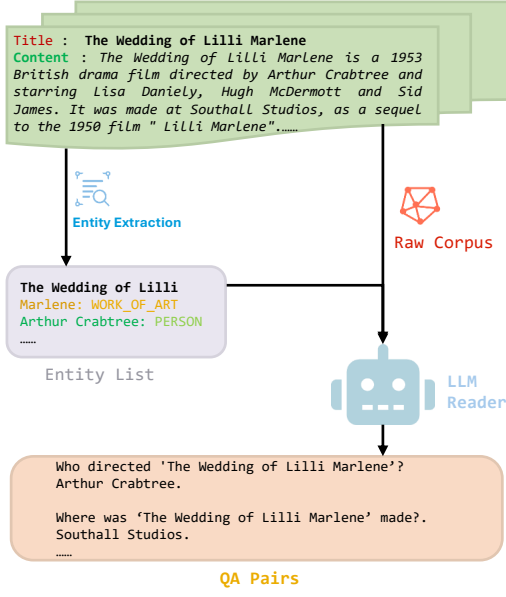


Figure 1: Overview of the Offline Knowledge Construction in COMPACTRAG. The raw corpus is first processed by an LLM “Reader” that reformulates document content into a set of atomic QA pairs. Each QA pair captures a minimal factual unit, annotated with entity information to ensure semantic completeness and prevent redundancy.

q_j . The decomposition is performed by an LLM once during inference initialization, and the dependency graph guides the iterative retrieval pipeline.

3.2.2 Answer Extractor. The Answer Extractor is responsible for identifying the correct entity or fact from retrieved QA pairs that correspond to a given sub-question q_i . Given q_i and its retrieved candidate QA pairs $\mathcal{P}_i = \{(q_{i,k}, a_{i,k})\}$, the extractor predicts the start and end positions of the correct answer span within the text context. The learning objective is a span prediction loss defined as:

$$\mathcal{L}_{\text{extract}} = -\frac{1}{N} \sum_{i=1}^N (\log P(s_i | q_i, C_i) + \log P(e_i | q_i, C_i)), \quad (1)$$

where C_i denotes the concatenation of candidate QA pairs, and s_i, e_i are the gold start and end token indices.

Training Data. To construct supervision for the extractor, we sample source passages from the training splits of the benchmarks used in this paper. For each passage, an LLM is prompted to generate sub-questions and corresponding correct and distractor QA pairs. The correct answer span is explicitly marked within the gold QA pair, while distractors introduce realistic retrieval noise.

Training Example of Answer Extractor

Sub-question: “Which country is the Eiffel Tower located in?”

QA pairs:

- (1) “Where is the Eiffel Tower situated?” **[gold]**
“Paris, France”
- (2) “What is the height of the Eiffel Tower?” “324 meters” [distractor]
- (3) “Which city hosts the Colosseum?” “Rome, Italy” [distractor]

Target span: “France”

The model learns to select the precise supporting evidence under noisy retrieval conditions.

3.2.3 Sub-Question Rewriter. As sub-questions may contain ambiguous or coreferential expressions (e.g., pronouns such as “he” or “it”), the Sub-Question Rewriter reformulates the current sub-question q_{i+1} by explicitly grounding it with the answer entity extracted from the preceding sub-question a_i . This mechanism ensures entity continuity across reasoning hops and prevents semantic drift during multi-hop retrieval.

Training Data. The data construction process mirrors the extractor setup. For each sample, the LLM generates an ambiguous question, an entity that resolves the ambiguity, and a corresponding rewritten form. To enhance robustness, additional samples are created through controlled perturbations such as entity masking and pronoun insertion.

Training Example of Question Rewrite

Ambiguous question: “Where was he born?”

Entity (from previous answer): “Albert Einstein”

Rewritten question: “Where was Albert Einstein born?”

The rewriter is trained using a conditional generation objective:

$$\mathcal{L}_{\text{rewrite}} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P(w_{i,t} | w_{i,<t}, q_{\text{amb},i}, e_i), \quad (2)$$

where e_i denotes the grounding entity and $w_{i,t}$ are the tokens of the target rewritten question. Teacher forcing is employed to stabilize sequence generation.

3.2.4 Synthesis Reasoning. After all sub-questions have been resolved and their supporting QA pairs collected, the system aggregates the retrieved knowledge and dependency chain. The final LLM call takes as input:

$$\{Q, \{q_i, a_i, \mathcal{P}_i\}_{i=1}^n\}$$

and generates the final answer through holistic reasoning. This single synthesis step completes the inference process, ensuring that the total number of LLM calls remains constant—two per query, regardless of hop count.

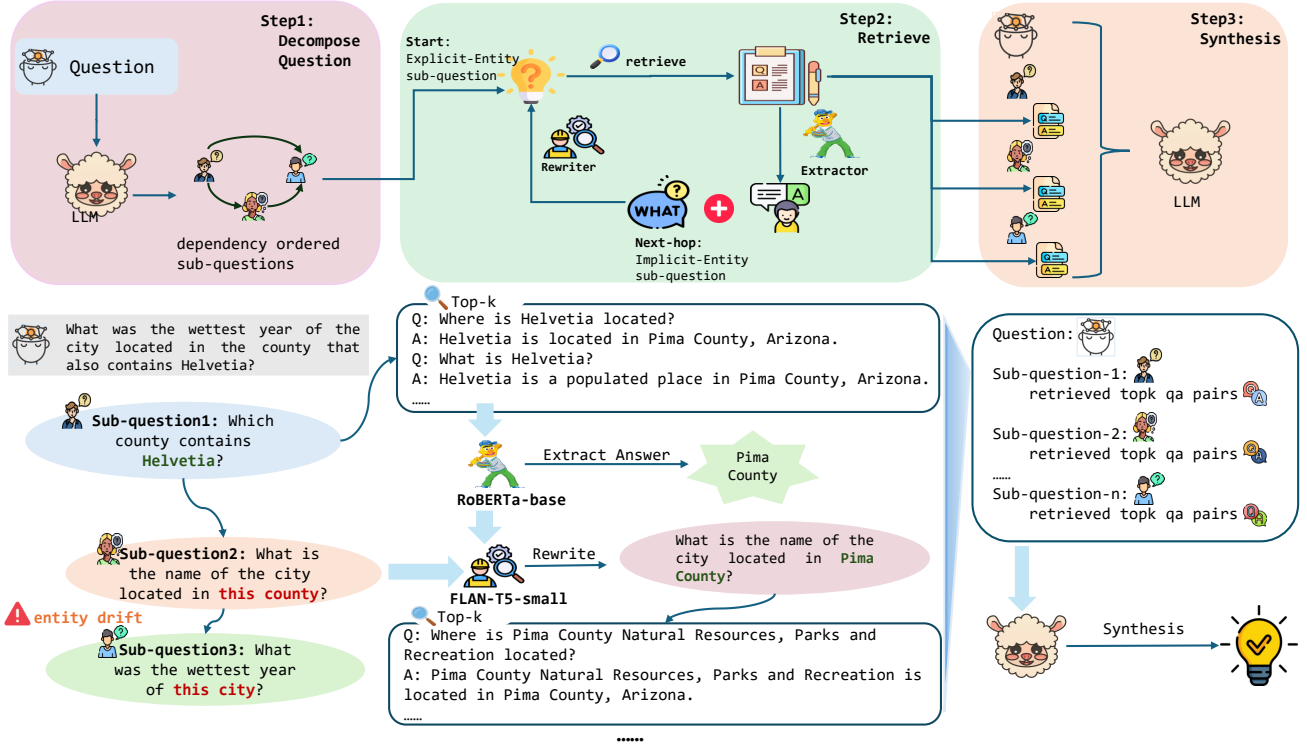


Figure 2: Overview of the Online Reasoning Pipeline in COMPACTRAG. The framework begins with query decomposition, where a complex multi-hop question is decomposed into dependency ordered sub-questions. Each sub-question is resolved through iterative retrieval over the atomic QA knowledge base, followed by lightweight answer extraction and question rewriting modules that ensure entity continuity and semantic grounding. Once all sub-questions are resolved, the retrieved QA pairs are aggregated and passed to a final synthesis reasoning step, completing the inference process with only two LLM calls per query.

3.3 Inference Integration

The full online reasoning workflow proceeds as follows:

- (1) Decompose the complex query Q into dependency-ordered sub-questions $\{q_1, q_2, \dots, q_n\}$.
- (2) For each q_i , retrieve candidate QA pairs $\mathcal{P}_i$ from the atomic QA knowledge base.
- (3) Run the *Answer Extractor* on $(q_i, \mathcal{P}_i)$ to obtain the grounded entity or answer a_i .
- (4) Use a_i to rewrite the next sub-question q_{i+1} via the *Sub-Question Rewriter*, obtaining q_{i+1}^{rew} .
- (5) Continue until all sub-questions are resolved; aggregate all evidence for final LLM synthesis reasoning.

This modular pipeline effectively decouples retrieval from LLM reasoning, maintaining accuracy and grounding while achieving significant reductions in token consumption and LLM invocations.

4 Experiment Setup

4.1 Benchmarks

We evaluate CompactRAG on three widely used multi-hop question answering benchmarks: HOTPOTQA [37], 2WIKIMULTIHOPQA [9], and the answerable subset of MuSiQUE [34]. For HOTPOTQA, we adopt the *distractor setting*, where each question is paired with ten

Wikipedia paragraphs, two containing gold supporting facts and eight serving as distractors. For 2WIKIMULTIHOPQA and MuSiQUE, which were originally designed for reading comprehension or mixed settings, we repurpose their associated contexts as the retrieval corpus to fit our evaluation framework. Illustrative examples from each benchmark are shown in Table 1.

Due to computational constraints and the substantial cost of LLM inference, we uniformly sample 250 questions from the development set of each dataset for evaluation. Sampling is performed while preserving the original distribution of question types and reasoning difficulty levels to ensure statistical representativeness. The sampled questions constitute our test set, and all corresponding contexts are included in the retrieval corpus used during inference.

4.2 Evaluation Metrics

We evaluate our approach from both *accuracy* and *efficiency* perspectives. For answer correctness, three complementary metrics are employed: **Exact Match (EM)**, **F1**, and **LLM-based Accuracy (Acc)**. **EM** measures the percentage of predictions that exactly match the gold answer string. **F1** captures the token level overlap between the prediction and the reference, balancing precision and recall. However, lexical metrics may underestimate semantically correct responses. To address this, we further adopt **LLM-based**

Table 1: Examples from the three multi-hop QA benchmarks used in our experiments. Each question requires reasoning over multiple Wikipedia paragraphs to arrive at the final answer.

Benchmark	Example Question and Reasoning Description
HOTPOTQA	Example: “Were both the film <i>Twelve Monkeys</i> and the TV series it inspired produced by the same company?” Reasoning: <i>The model must first find that Twelve Monkeys (film) was produced by Universal Pictures, then check the producer of the TV adaptation, verifying both were indeed produced by the same studio.</i>
2WIKIMULTIHOPQA	Example: “Who was born earlier, the author of <i>Pride and Prejudice</i> or the composer of <i>The Magic Flute</i> ?” Reasoning: <i>The model needs to identify that Jane Austen wrote Pride and Prejudice and Wolfgang Amadeus Mozart composed The Magic Flute, then compare their birth years.</i>
MUSIQUE	Example: “Which actor who played a character named Jack also starred in the film <i>The Departed</i> ?” Reasoning: <i>Requires multi-step reasoning: find that Jack Dawson was played by Leonardo DiCaprio in Titanic, then confirm DiCaprio also starred in The Departed.</i>

Accuracy (Acc), in which a strong evaluator LLM assesses whether the predicted answer is semantically consistent with the reference answer, prompt shown as in Appendix B :

Beyond correctness, we also report the average token consumption per query, counting both input and output tokens during inference. This efficiency metric directly reflects computational and monetary cost under real-world deployment, and demonstrates the advantage of our method in reducing redundancy and improving inference efficiency.

4.3 Baselines

To evaluate COMPACTRAG, we compare it against representative retrieval-generation frameworks for multi-hop reasoning, as well as a vanilla RAG baseline.

Vanilla RAG. A standard retrieval-augmented generation pipeline that retrieves the top- k passages using the original multi-hop question, followed by a single LLM call for answer generation. This simple one-shot approach lacks explicit reasoning decomposition and serves to highlight the benefits of iterative reasoning and structured query decomposition.

Self-Ask. [24]. A prompting-based method that enhances chain-of-thought reasoning by allowing the model to ask and answer intermediate questions before producing the final answer. In our implementation, the original search engine is replaced with our retriever for consistent retrieval conditions.

IRCoT. [35]. An interleaved reasoning-retrieval method that alternates between chain-of-thought generation and retrieval. Each reasoning step guides retrieval toward relevant evidence, while retrieved content refines subsequent reasoning, enabling progressive evidence accumulation.

Iter-RetGen. [28]. A recent iterative retrieval-generation framework. At each step, the model generates a partial response from the current context, identifies information gaps or unresolved entities, and converts them into new retrieval queries. Retrieved passages are appended, and the model updates its response. We use 4 iterations, following the original paper’s observation that performance saturates after 4 steps.

All methods use the same retrieval corpus and retriever. At each step, the top-5 passages by similarity score are selected as evidence for subsequent reasoning.

4.4 Models

We use LLAMA3.1-8B [1] as the main LLM for all baselines and COMPACTRAG, with decoding temperature set to 0 for deterministic inference.

The *Answer Extractor* is based on ROBERTA-BASE [20] (125M parameters) and identifies answer spans from retrieved QA pairs. The *Sub-Question Rewriter* uses FLAN-T5-SMALL [4] (80M parameters) to rewrite ambiguous sub-questions by explicitly inserting resolved entities. Both modules are lightweight, enabling local reasoning without invoking the main LLM.

Training data for these modules are generated with GPT-4 [22] at temperature 0 to ensure precise supervision. For dense retrieval, we adopt CONTRIEVER [10], an unsupervised contrastive dense retriever that encodes questions and passages into a shared semantic space, providing robust zero-shot retrieval across domains.

For an upper-bound comparison, we include a variant of COMPACTRAG where the offline atomic QA knowledge base is constructed directly from the corpus using GPT-4, serving as a reference for evaluating the quality of generated atomic QA knowledge.

5 Results and Analysis

In this section, we present and analyze the experimental results of COMPACTRAG and competing baselines across three multi-hop QA benchmarks: HOTPOTQA, 2WIKIMULTIHOPQA, and MUSIQUE. We report Exact Match (EM), F1, and accuracy (Acc) scores, along with the average token consumption per query. Results are summarized in Table 2.

5.1 Overall Performance

Table 2 presents the main experimental results, comparing two configurations of COMPACTRAG against several baseline methods. To ensure a controlled evaluation, all methods utilize the same retrieval setup. Our primary comparison employs the same backbone LLM LLAMA-3.1-8B across methods. Under this setting, COMPACTRAG achieves competitive performance in accuracy on the multi-hop benchmarks HotpotQA, 2WikiMultiHopQA, and MuSiQue. Notably, it attains this performance while consuming significantly fewer tokens per query than iterative baselines. This result underscores the algorithmic advantage of our approach, which reorganizes the corpus into atomic QA pairs and decouples retrieval from LLM reasoning.

Table 2: Main results on multi-hop QA benchmarks. All methods share identical retrieval settings for fair comparison. “Token / Sample” denotes the average total tokens consumed (prompt + generation) per query during inference. Best results are bolded.

Method	HotpotQA			2WikiMultiHopQA			MuSiQue			Token / Sample
	EM	F1	Acc	EM	F1	Acc	EM	F1	Acc	
Vanilla-RAG	27.60	30.32	50.80	20.80	24.38	72	5.600	11.36	8.400	2.7K
Self-Ask	23.60	26.30	40.80	27.60	33.08	34.40	19.60	28.34	24.80	6.9K
IRCoT	42.80	48.95	65.20	42.80	48.99	48.80	21.20	29.08	32.40	10.2K
Iter-RetGen	<u>46.80</u>	52.24	<u>72.40</u>	50.80	59.73	61.20	24.80	32.42	40.00	4.7K
CompactRAG										
LLaMA-3.1-8b-Reading (Ours)	45.20	<u>66.21</u>	70.40	40.40	49.62	53.20	<u>26.80</u>	<u>37.63</u>	<u>41.20</u>	1.9K
CompactRAG										
GPT-4-Reading (Ours)	49.60	69.54	77.20	<u>47.20</u>	<u>55.67</u>	<u>57.20</u>	30.80	42.34	43.60	1.9K

To explore the upper performance bound of our system, we further evaluate a version where the atomic QA knowledge base is constructed offline using the more powerful GPT-4 model. This configuration leads to improved accuracy, as shown in Table 2. It is important to clarify that this preprocessing step incurs a one-time. The online inference stage still remains fully based on LLAMA3.1-8B.

In summary, these findings demonstrate that COMPACTRAG delivers a favorable balance of efficiency and accuracy even with a middle scale LLM. Furthermore, the system’s accuracy exhibits potential for further enhancement through improvements in the quality of the underlying atomic QA knowledge base.

5.2 Token Efficiency Analysis

We further evaluate the token efficiency of COMPACTRAG in comparison with several iterative retrieval–reasoning baselines. Figure 3 shows the cumulative token consumption on HOTPOTQA (others are shown in Appendix C) as the number of user queries increases. Because COMPACTRAG includes an offline preprocessing to construct the atomic QA knowledge base, it incurs an initial token cost before online inference begins. However, as the number of user requests grows, the cumulative cost curve of COMPACTRAG increases at a much slower rate than those of iterative RAG baselines such as SELF-ASK, IRCoT, and ITER-RETGEN. The initial offline expense is quickly amortized, and the overall token usage remains substantially lower than that of the iterative methods. This result demonstrates the long-term efficiency of COMPACTRAG, particularly in deployment scenarios involving large volumes of user interactions.

To provide a more granular view, Figure 4 plots the token consumption per user query. Here, the horizontal axis corresponds to the sequence of user queries (each representing a distinct multi-hop question), and the vertical axis denotes the total tokens consumed to resolve the query. The observed fluctuations are primarily due to differences in question complexity, queries requiring deeper reasoning or more hops naturally consume more tokens. Nonetheless,

COMPACTRAG consistently maintains a much lower average token cost across queries compared to iterative baselines. This stability results from its fixed two-call design and compact QA retrieval, which together eliminate redundant LLM invocations while preserving reasoning completeness.

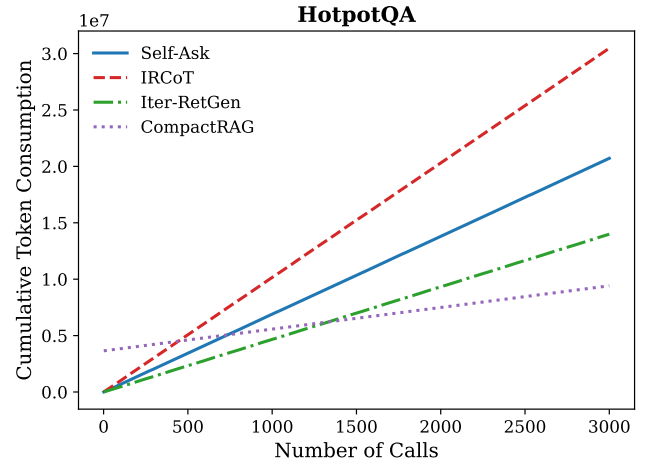


Figure 3: Cumulative token consumption on HOTPOTQA. Although COMPACTRAG incurs an initial offline cost to construct the atomic QA knowledge base, its cumulative token usage grows slowly and eventually remains well below that of iterative baselines as user queries accumulate.

5.3 Ablation Study

To evaluate the contribution of the *Answer Extractor* and *Sub-Question Rewriter* modules, we conduct two ablation experiments using the LLAMA3.1-8B-based QA knowledge base. All configurations adopt identical retrieval settings and inference procedures.

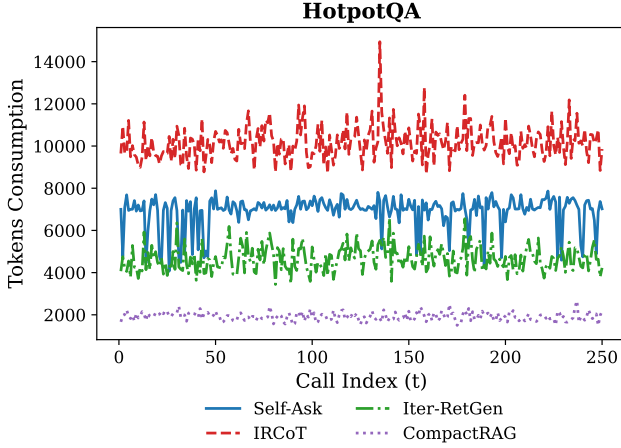


Figure 4: Per-query token consumption on HotpotQA. Each point represents one user query (a multi-hop question). The token cost varies with question complexity, leading to oscillations across the curve. Despite this variation, COMPACTRAG maintains consistently lower per-query consumption than iterative baselines, reflecting its efficiency and stability in online inference.

Table 3: Ablation results (Accuracy %) on three benchmarks using the LLAMA3.1-8B QA knowledge base.

Method	HotpotQA	2WikiMultiHopQA	MuSiQue
CompactRAG (Full)	70.4	53.2	41.2
w/o Rewriter	63.2	48.8	35.8
w/o Extractor & Rewriter	58.4	44.2	32.6

- **w/o Rewriter:** The rewriter module is removed. The extracted answer is directly concatenated with the next sub-question, encoded by CONTRIEVER, and used to retrieve QA pairs.
- **w/o Extractor & Rewriter:** Both modules are removed. The raw sub-questions generated by the LLM decomposition are encoded by CONTRIEVER and used directly for retrieval without any local reasoning.

Table 3 reports the accuracy across three benchmarks. Removing either component leads to a consistent decline in performance, confirming that both modules are essential for maintaining entity grounding and retrieval precision. The degradation is particularly evident when the rewriter is removed, indicating that explicit entity resolution is critical for accurate multi-hop reasoning.

These results demonstrate that both the extractor and rewriter modules significantly enhance COMPACTRAG’s ability to preserve contextual consistency and reasoning accuracy while keeping inference efficient with minimal LLM calls.

5.4 Discussion

The experimental findings collectively highlight the efficiency and accuracy trade-off addressed by COMPACTRAG. Unlike prior iterative RAG systems, which repeatedly alternate between retrieval

and LLM reasoning, our design constrains the number of LLM invocations to two per query while maintaining competitive accuracy. This fixed call structure not only reduces inference cost but also simplifies the overall pipeline, making it more predictable and scalable for real world deployment. The results also reveal that the quality of the atomic QA knowledge base plays a crucial role in downstream reasoning. When the QA base is constructed using a stronger reader such as GPT-4, accuracy improves across all benchmarks, demonstrating that enhancing the semantic fidelity of offline knowledge can directly boost reasoning quality at inference time. However, even with a smaller LLM such as LLAMA3.1-8B, COMPACTRAG achieves comparable performance to much heavier iterative methods, underscoring the robustness of its design.

From a broader perspective, these results suggest that efficient reasoning in retrieval-augmented systems does not necessarily require larger models or more frequent model calls. Instead, structuring external knowledge into concise, semantically aligned units and leveraging lightweight reasoning components can yield comparable accuracy with drastically lower computational overhead. This insight opens a promising direction for future research on scalable, cost-efficient.

6 Conclusion

This paper introduced COMPACTRAG, a retrieval-augmented generation framework designed to achieve efficient multi-hop reasoning with minimal LLM usage. By decoupling retrieval and reasoning through the construction of an atomic QA knowledge base and the integration of lightweight reasoning modules, COMPACTRAG reduces token consumption and stabilizes inference cost regardless of question complexity. Unlike iterative RAG methods that repeatedly invoke LLMs, our approach fixes the number of calls to two per query while maintaining competitive accuracy across multiple benchmarks. Extensive experiments on HOTPOTQA, 2WIKIMULTIHOPQA, and MUSIQUE demonstrate that COMPACTRAG significantly lowers computational overhead without sacrificing answer quality. Ablation studies further confirm the complementary roles of the *Answer Extractor* and *Sub-Question Rewriter*, while additional analysis show that improving the semantic quality of the atomic QA base can further enhance performance.

Overall, COMPACTRAG highlights a promising direction for developing cost-efficient and scalable RAG systems. By combining modular reasoning, efficient retrieval, and pre-processed knowledge, it offers a practical blueprint for large-scale multi-hop reasoning tasks. Future work will explore adaptive retrieval strategies, dynamic sub-question generation, and cross-domain generalization of QA knowledge bases, extending the framework to broader open-domain reasoning, interactive dialogue, and knowledge-intensive NLP applications.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62306138), the Jiangsu Natural Science Foundation (Grant No. BK20230784), and the Innovation Program of the State Key Laboratory for Novel Software Technology at Nanjing University (Grant Nos. ZZKT2024B15 and ZZKT2025B25).

References

- [1] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*. PMLR, 2206–2240.
- [3] Rong Cheng, Jinyi Liu, Yan Zheng, Fei Ni, Jiazhen Du, Hangyu Mao, Fuzheng Zhang, Bo Wang, and Jianye Hao. 2025. DualRAG: A Dual-Process Approach to Integrate Reasoning and Retrieval for Multi-Hop Question Answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 31877–31899. doi:10.18653/v1/2025.acl-long.1539
- [4] Hyung Won Chung and others. 2022. Scaling Instruction-Finetuned Language Models. doi:10.48550/ARXIV.2210.11416
- [5] Jinyuan Fang, Zaiqiao Meng, and Craig MacDonald. 2024. TRACE the Evidence: Constructing Knowledge-Grounded Reasoning Chains for Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 8472–8494. doi:10.18653/v1/2024.findings-emnlp.496
- [6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL] <https://arxiv.org/abs/2312.10997>
- [7] Hanghui Guo, Jia Zhu, Shimin Di, Weijie Shi, Zhangze Chen, and Jiajie Xu. 2025. DioR: Adaptive Cognitive Detection and Contextual Retrieval Optimization for Dynamic Retrieval-Augmented Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 2953–2975. doi:10.18653/v1/2025.acl-long.148
- [8] Wangzhen Guo, Qinkang Gong, Yanghui Rao, and Hanjiang Lai. 2023. Counterfactual Multi-hop QA: A Cause-Effect Approach for Reducing Disconnected Reasoning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 4214–4226. doi:10.18653/v1/2023.acl-long.231
- [9] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Barcelona, Spain (Online), 6609–6625. <https://www.aclweb.org/anthology/2020.coling-main.580>
- [10] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised Dense Information Retrieval with Contrastive Learning. doi:10.48550/ARXIV.2112.09118
- [11] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: few-shot learning with retrieval augmented language models. *J. Mach. Learn. Res.* 24, 1, Article 251 (Jan. 2023), 43 pages.
- [12] Yuelyu Ji, Rui Meng, Zhuochun Li, and Daqing He. 2025. Curriculum Guided Reinforcement Learning for Efficient Multi Hop Retrieval Augmented Generation. arXiv:2505.17391 [cs.CL] <https://arxiv.org/abs/2505.17391>
- [13] Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 7969–7992. doi:10.18653/v1/2023.emnlp-main.495
- [14] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 59532–59569.
- [15] Yiqiao Jin, Kartik Sharma, Vineeth Rakesh, Yingdong Dou, Menghai Pan, Mahashweta Das, and Srikanth Kumar. 2025. SARA: Selective and Adaptive Retrieval-Augmented Generation with Context Compression. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*. <https://openreview.net/forum?id=7qSlrCYtI>
- [16] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [18] Ruosen Li, Zimu Wang, Son Quoc Tran, Lei Xia, and Xinya Du. 2025. MEQA: a benchmark for multi-hop event-centric question answering with explanations. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 4028, 28 pages.
- [19] Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. 2025. HopRAG: Multi-Hop Reasoning for Logic-Aware Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 1897–1913. doi:10.18653/v1/2025.findings-acl.97
- [20] Yinhan Liu, Mylène Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR* abs/1907.11692 (2019). arXiv:1907.11692 <http://arxiv.org/abs/1907.11692>
- [21] Man Luo, Xin Xu, Zhuyun Dai, Panupong Pasupat, Mehran Kazemi, Chitta Baral, Vaiva Imbrasaitė, and Vincent Y Zhao. 2023. Dr.ICL: Demonstration-Retrieved In-context Learning. arXiv:2305.14128 [cs.CL] <https://arxiv.org/abs/2305.14128>
- [22] OpenAI et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [23] Ethan Perez, Patrick Lewis, Wen-tau Yih, Kyunghyun Cho, and Douwe Kiela. 2020. Unsupervised Question Decomposition for Question Answering. arXiv:2002.09758 [cs.CL] <https://arxiv.org/abs/2002.09758>
- [24] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah Smith, and Mike Lewis. 2023. Measuring and Narrowing the Compositionality Gap in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 5687–5711. doi:10.18653/v1/2023.findings-emnlp.378
- [25] Zexuan Qiu, Zijing Ou, Bin Wu, Jingjing Li, Aiwei Liu, and Irwin King. 2025. Entropy-Based Decoding for Retrieval-Augmented Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 4616–4627. doi:10.18653/v1/2025.naacl-long.236
- [26] Vipula Rawte, Rajarshi Roy, Gurpreet Singh, Danush Khanna, Yaswanth Narasupalli, Basab Ghosh, Abhay Gupta, Argha Kamal Samanta, Aditya Shingote, Aadi Krishna Vikram, Vinija Jain, Aman Chadha, Amit Sheth, and Amitava Das. 2025. RADIANT: Retrieval Augmented Entity-context Alignment – Introducing RAG-ability and Entity-Context Divergence. arXiv:2507.02949 [cs.CL] <https://arxiv.org/abs/2507.02949>
- [27] Ahmad OM Saleh, Gokhan Tur, and Yucel Saygin. 2025. SG-RAG MOT: Sub-Graph Retrieval Augmented Generation with Merging and Ordering Triplets for Knowledge Graph Multi-Hop Question Answering. *Machine Learning and Knowledge Extraction* 7, 3 (2025), 74.
- [28] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Enhancing Retrieval-Augmented Large Language Models with Iterative Retrieval-Generation Synergy. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 9248–9274. doi:10.18653/v1/2023.findings-emnlp.620
- [29] Yucheng Shi, Qiaoyu Tan, Xuansheng Wu, Shaochen Zhong, Kaixiong Zhou, and Ninghao Liu. 2024. Retrieval-enhanced knowledge editing in language models for multi-hop question answering. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2056–2066.
- [30] Zhengliang Shi, Shuo Zhang, Weiwei Sun, Shen Gao, Pengjie Ren, Zhumin Chen, and Zhaochun Ren. 2024. Generate-then-Ground in Retrieval-Augmented Generation for Multi-hop Question Answering. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 7339–7353. doi:10.18653/v1/2024.acl-long.397
- [31] Weihang Su, Yichen Tang, Qingyao Ai, Zhiqing Wu, and Yiqun Liu. 2024. DRAGIN: Dynamic Retrieval Augmented Generation based on the Real-time Information Needs of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 12991–13013. doi:10.18653/v1/2024.acl-long.702
- [32] Hexiang Tan, Fei Sun, Wanli Yang, Yuanzhao Wang, Qi Cao, and Xueqi Cheng. 2024. Blinded by Generated Contexts: How Language Models Merge Generated and Retrieved Contexts When Knowledge Conflicts?. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6207–6227.
- [33] Yixuan Tang and Yi Yang. 2024. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. arXiv:2401.15391 [cs.CL]

- [34] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* (2022).
- [35] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 10014–10037. doi:10.18653/v1/2023.acl-long.557
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1800, 14 pages.
- [37] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 2369–2380. doi:10.18653/v1/D18-1259
- [38] Zijun Yao, Weijian Qi, Liangming Pan, Shulin Cao, Linmei Hu, Liu Weichuan, Lei Hou, and Juanzi Li. 2025. SeaKR: Self-aware Knowledge Retrieval for Adaptive Retrieval Augmented Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 27022–27043. doi:10.18653/v1/2025.acl-long.1312
- [39] Linhao Ye, Lang Yu, Zhikai Lei, Qin Chen, Jie Zhou, and Liang He. 2025. Optimizing Question Semantic Space for Dynamic Retrieval-Augmented Multi-hop Question Answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 17814–17824. doi:10.18653/v1/2025.acl-long.871
- [40] Zhuocheng Zhang, Yang Feng, and Min Zhang. 2025. LevelRAG: Enhancing Retrieval-Augmented Generation with Multi-hop Logic Planning over Rewriting Augmented Searchers. arXiv:2502.18139 [cs.CL] <https://arxiv.org/abs/2502.18139>
- [41] Zexuan Zhong, Zhengxuan Wu, Christopher Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing Knowledge Editing in Language Models via Multi-Hop Questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 15686–15702. doi:10.18653/v1/2023.emnlp-main.971
- [42] Rongzhi Zhu, Xiangyu Liu, Zequn Sun, Yiwei Wang, and Wei Hu. 2025. Mitigating Lost-in-Retrieval Problems in Retrieval Augmented Multi-Hop Question Answering. In *ACL*.
- [43] Ziyuan Zhuang, Zhiyang Zhang, Sitao Cheng, Fangkai Yang, Jia Liu, Shujian Huang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2024. EfficientRAG: Efficient Retriever for Multi-Hop Question Answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 3392–3411. doi:10.18653/v1/2024.emnlp-main.199

A QAs Generation Prompt

Generate QAs Prompt

System Role: Knowledge extraction and question generation system.

Task Description: You will receive:

- (1) Original passage text
 - (2) Extracted entities and relationships from the passage
- Your task is to generate atomic knowledge facts and corresponding QA pairs.

Output Format: A single JSON object only (nothing else) enclosed in a “`json ... `” code block with exactly two keys:

- "atomic_facts": an array of atomic knowledge facts (each fact should be independent, complete, and self-contained)
- "qa": an array of objects {"question": "...", "answer": "..."}.

Rules for Atomic Facts:

- (1) Each fact should be an independent, complete statement that can stand alone.
- (2) Cover both explicit and implicit relationships mentioned in the text.
- (3) Include background knowledge and context that help understand the entities.
- (4) Each fact should be concise but informative (preferably one sentence).
- (5) Do not duplicate facts or add information not present in the passage.

Rules for QA Generation:

- (1) Each question must be short (≤ 12 words) and start with a question word (*Who, What, When, Where, Which, How, How many*).
- (2) Use explicit entity names from the entities list; avoid pronouns or vague references.
- (3) Each answer must be an exact verbatim substring from the original passage.
- (4) Ensure coverage of all important entities and relationships.
- (5) Avoid duplicate questions or answers.

Example:

[Original Text]:

Lilli's Marriage (German: Lillis Ehe) is a 1919 German silent film directed by Jaap Speyer. It is a sequel to the film "Lilli", and premiered at the Marmorhaus in Berlin. The film's art direction was by Hans Dreier.

[Entity List]:

Lilli's Marriage (WORK_OF_ART), Lillis Ehe (WORK_OF_ART), Jaap Speyer (PERSON), Lilli (WORK_OF_ART), Marmorhaus in Berlin (FAC), Hans Dreier (PERSON), 1919 (DATE)

[Output JSON]:

```
```json
{
 "atomic_facts": [
 "Lilli's Marriage is a 1919 German silent film",
 "Lilli's Marriage is also known as Lillis Ehe in German",
 "Jaap Speyer directed Lilli's Marriage",
 "Lilli's Marriage is a sequel to the film Lilli",
 "Lilli's Marriage premiered at the Marmorhaus in Berlin",
 "Hans Dreier was responsible for the art direction of Lilli's Marriage",
 "Lilli's Marriage was released in 1919"
],
 "qa": [
 {"question": "What is Lilli's Marriage?", "answer": "a 1919 German silent film"},
 {"question": "Who directed Lilli's Marriage?", "answer": "directed by Jaap Speyer"},
 {"question": "Which film is Lilli's Marriage a sequel to?", "answer": "It is a sequel to the film \"Lilli\""},
 {"question": "Where did Lilli's Marriage premiere?", "answer": "premiered at the Marmorhaus in Berlin"},
 {"question": "Who was responsible for the art direction of Lilli's Marriage?", "answer": "Hans Dreier"}
]
}
```

Now process the following passage:

[Original Text]:

{passage}

[Entity List]:

{entity\_info}

## B Prompt-evaluate answer based LLM

### Prompt-evaluate answer based LLM

You are an experienced linguist who is responsible for evaluating the correctness of the generated responses. You are provided with question, the generated responses and the corresponding ground truth answer. Your task is to compare the generated responses with the ground truth responses and evaluate the correctness of the generated responses. Response directly “yes” or “no”.

Question: {question}  
 Prediction: {prediction}  
 Ground-truth Answer: {answer}  
 Your response:

## C Additional Token Efficiency Results on Other Benchmarks

To further validate the token efficiency of COMPACTRAG, we report additional analyses on the 2WIKIMULTIHOPQA and MUsIQUE benchmarks. Each benchmark includes two visualizations: cumulative token consumption and per-query token consumption. Both exhibit the same trends as observed on HOTPOTQA—an initial offline cost for building the QA knowledge base, followed by sustained efficiency during online inference. These consistent patterns demonstrate that COMPACTRAG maintains its efficiency advantage across different datasets and reasoning complexities.

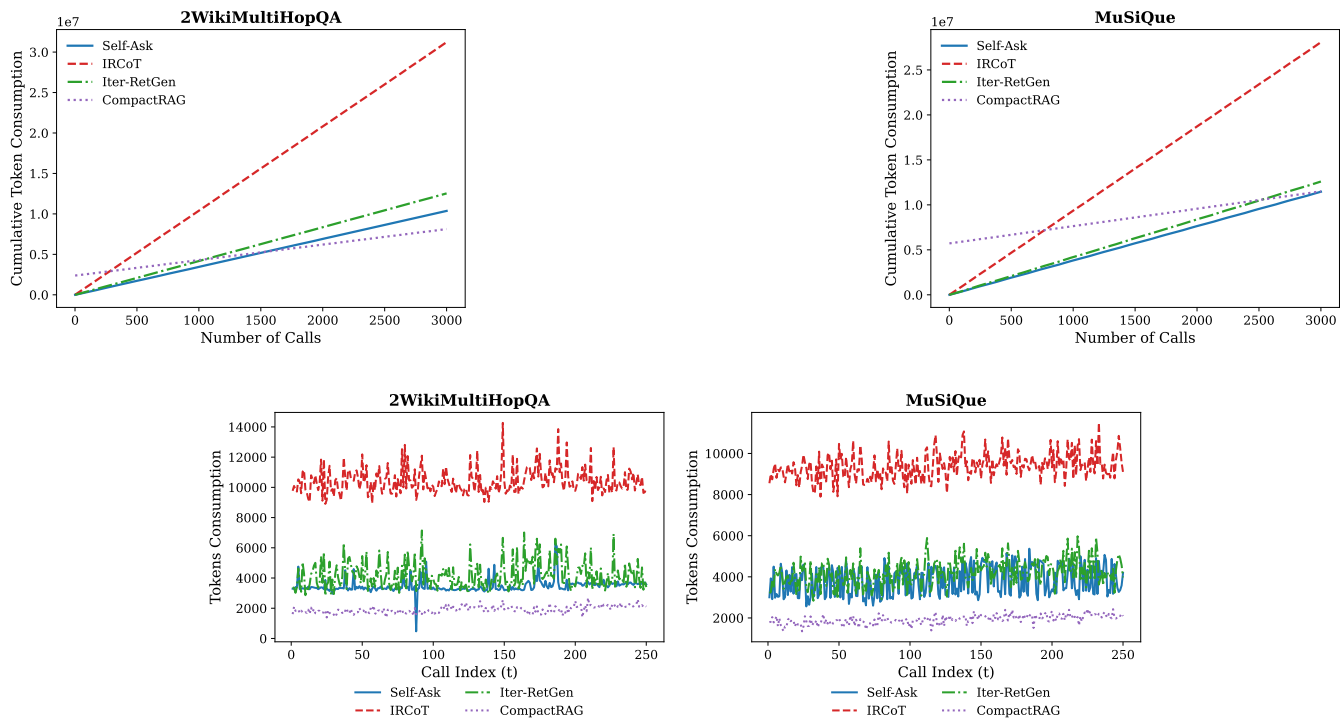


Figure 5: Token consumption comparison across additional benchmarks.